

UNIVERSIDAD PERUANA DE CIENCIAS APLICADAS

FACULTAD DE INGENIERÍA

CURSO: 1AMA0726 – MATEMÁTICA COMPUTACIONAL

FLUJO MÁXIMO MEDIANTE EL ALGORITMO DE FORD-FULKERSON

Informe final del proyecto

INTEGRANTES

Apellidos y nombres	**Código**
Aranda Eugenio, Lincoln Angel	u20251g186
Chavez Bendezu, Alexandro Aldair	U20251E715
Chura Chata, Yhon Eduardo	U20251G281
De la Cruz Huamani, Oliver Vidal	u20251g950
Ortiz Saavedra, Sebastian Osvaldo	U20251G510

PROFESOR

Juan Manuel Matos Quevedo

UPC, noviembre de 2026

Contenido

Introducción 3
Objetivo 4
Metodología 4
Algoritmo de Ford-Fulkerson 4
Orígenes y destinos ficticios 5
Verificación mediante el corte mínimo 5
Desarrollo del código 6
Librerías Utilizadas 6
Descripción General del Programa 6
Detección de Fuentes y Sumideros e Inyección de Nodos Ficticios 7
Búsqueda del Camino Aumentante mediante BFS 8
Cálculo y Validación del Cuello de Botella 9
Cálculo del Corte Mínimo 10
Diagrama de Flujo del Programa Ejecutable 12
Visualización de Resultados 13
Programa 17
Resultados 18
Interpretación y Representación 18
Cálculo 18
Análisis y Argumentación 19
Conclusiones 20
Referencias Bibliográficas 21

Flujo máximo mediante el algoritmo de Ford-Fulkerson

Introducción

En el mundo moderno, la gestión y distribución eficiente de recursos constituye un desafío fundamental en áreas como las telecomunicaciones, la logística de transporte, el suministro de energía y el tráfico de datos en redes informáticas. En todos estos escenarios, la infraestructura cuenta con canales o vías de comunicación que poseen limitaciones físicas de capacidad, lo que plantea la necesidad constante de determinar la cantidad óptima o máxima de flujo que puede trasladarse de un punto de origen a un destino sin saturar la red.

Para dar solución formal a esta problemática, la matemática computacional y la teoría de grafos han desarrollado modelos y algoritmos clásicos de optimización de redes, entre los cuales destaca el método propuesto por L. R. Ford y D. R. Fulkerson en la década de 1950. El análisis de estas estructuras permite no solo hallar el rendimiento máximo de una red, sino también identificar cuellos de botella y cortes críticos indispensables para la toma de decisiones técnicas y operativas. Además, muchas redes reales poseen varios puntos de origen y varios puntos de destino, por lo que resulta necesario transformarlas en una red equivalente con una única fuente y un único sumidero mediante orígenes y destinos ficticios.

El presente informe expone la versión final de un programa web interactivo orientado al problema del flujo máximo. Se detalla la formulación lógica del algoritmo de Ford-Fulkerson, la arquitectura modular del código, la reducción de redes con múltiples fuentes y sumideros, los mecanismos interactivos que permiten simular paso a paso la construcción de la red residual y la verificación del resultado mediante el Teorema del Flujo Máximo y Corte Mínimo. Finalmente, se presentan los resultados obtenidos con un caso de prueba y las conclusiones del proyecto.

Objetivo

Desarrollar una aplicación web interactiva utilizando JavaScript, HTML5 y CSS que implemente el algoritmo de Ford-Fulkerson para resolver el problema del flujo máximo en grafos dirigidos, incluyendo redes con múltiples fuentes y sumideros mediante nodos ficticios, con la finalidad de automatizar el cálculo iterativo de capacidades residuales y proporcionar una herramienta didáctica y visual que permita identificar cuellos de botella y verificar el corte mínimo en redes de transporte y comunicaciones.

Metodología

Algoritmo de Ford-Fulkerson

El programa implementa el algoritmo de Ford-Fulkerson para resolver el problema de optimización de flujo máximo en redes de transporte y transmisión dirigidas. En primer lugar, el sistema solicita al usuario la cantidad de vértices de la red dentro del rango reglamentario entre 7 y 16, junto con la definición de las aristas y sus capacidades c(u, v) mediante una generación manual o aleatoria. A partir de estos datos, la aplicación inicializa la red asignando un flujo nulo a todos los arcos, f(e) = 0 para todo e ∈ E. Luego se establece la fuente s y el sumidero t; si la red posee varias fuentes o varios sumideros, el sistema los integra mediante una fuente y un sumidero ficticios.

Durante la resolución, el usuario selecciona interactivamente los caminos de aumento desde la fuente hasta el sumidero a través de aristas que cuenten con capacidad disponible, es decir, donde se cumpla c(u, v) − f(u, v) > 0. Este procedimiento equivale a la etiquetación de vértices estudiada en clase: la fuente recibe la etiqueta (−, ∞) y cada vértice alcanzado recibe la etiqueta (x^+, Δ(y)), donde x es el vértice anterior y Δ(y) es el flujo adicional que puede llegar hasta y. Una vez definida la trayectoria p, el sistema calcula el incremento de flujo permitido, que corresponde a la menor capacidad residual del camino:

$$\Delta = \min_{(u,v)\in p} \{\, c(u, v) - f(u, v) \,\}$$

Con este valor, el programa actualiza la red sumando Δ unidades al flujo de cada arista del camino, $f(u, v) \leftarrow f(u, v) + \Delta$, e incrementa el valor del flujo total, $|f| \leftarrow |f| + \Delta$. El procedimiento se repite hasta que el sumidero ya no puede ser alcanzado, momento en el que el flujo obtenido es máximo.

Orígenes y destinos ficticios

Cuando la red posee múltiples fuentes $\{s_1, s_2, \ldots, s_m\}$ y múltiples sumideros $\{t_1, t_2, \ldots, t_n\}$, el programa la transforma en una red equivalente con una única fuente y un único sumidero. Para ello, analiza los grados de cada vértice: un vértice con grado de entrada igual a cero y grado de salida positivo se considera fuente, y un vértice con grado de salida igual a cero y grado de entrada positivo se considera sumidero. Si se detectan dos o más fuentes, se agrega una fuente ficticia S conectada a cada fuente original mediante aristas de capacidad infinita; de manera análoga, si se detectan dos o más sumideros, se agrega un sumidero ficticio T al que se conectan todos los sumideros originales con capacidad infinita. Esta transformación no altera el flujo máximo de la red y permite aplicar directamente el algoritmo de Ford-Fulkerson entre S y T.

Verificación mediante el corte mínimo

Al agotar los caminos de aumento, el sistema construye el corte (S, T) agrupando en el conjunto S los vértices alcanzables desde la fuente en la red residual y en el conjunto T los vértices restantes. La capacidad del corte se obtiene sumando únicamente las capacidades de los arcos que van de S hacia T:

$$c(S, T) = \Sigma_{(u \in S,\, v \in T)}\, c(u, v)$$

Si el valor del flujo obtenido es igual a la capacidad del corte, $|f| = c(S, T)$, entonces, según el Teorema del Flujo Máximo y Corte Mínimo, el flujo encontrado es máximo y el corte hallado es mínimo.

Desarrollo del código

Librerías Utilizadas

Se emplearon únicamente las APIs nativas del navegador como motor de la aplicación:

- **DOM API:** para la captura de eventos e interacción con los controles del usuario.
- **SVG DOM API:** para la manipulación y el renderizado dinámico de nodos, aristas, marcadores de flecha y de la línea del corte mínimo.
- **ECMAScript (JavaScript ES6+):** para la lógica del grafo, las búsquedas BFS y DFS, el cálculo de capacidades residuales y el corte mínimo.

La elección de una solución nativa sin dependencias garantiza la ejecución inmediata en la web, sin entornos de compilación ni gestores de paquetes. Solo se incorpora la tipografía Inter desde Google Fonts para la interfaz.

Figura 1

Vinculación de la hoja de estilos y del script lógico principal

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Modelo de Red - Algoritmo de Ford-Fulkerson</title>
  ...
  <link rel="preconnect" href="https://fonts.googleapis.com">
  <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
  <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap" rel="stylesheet">
  <link rel="stylesheet" href="style.css">
</head>
  ...

  <script src="script.js"></script>
</body>
```

Nota. Estructura del documento HTML5 que enlaza la tipografía, la hoja de estilos en cascada (CSS) y el script principal en JavaScript nativo encargado de la lógica del algoritmo. Elaboración propia.

Descripción General del Programa

El sistema recibe la configuración de la red de flujos mediante una interfaz gráfica en la web. Después, la aplicación genera la representación visual del grafo dirigido en un lienzo

SVG, distribuye los vértices de izquierda a derecha según sus niveles y gestiona la asignación de capacidades en las aristas, con el objetivo de simular iterativamente el algoritmo de Ford-Fulkerson y determinar el flujo máximo de la red.

Figura 2

Ingreso y validación de aristas y capacidades de la red

```
// Inserta manualmente una arista dirigida y con capacidad en el grafo.
function conectarArista() {
  const origenVal = inputNodoInicial.value.trim();
  const destinoVal = inputNodoFinal.value.trim();
  const pesoVal = inputPeso.value.trim();

  const nodoOrig = obtenerNodo(origenVal);
  const nodoDest = obtenerNodo(destinoVal);

  if (!nodoOrig || !nodoDest) {
    mostrarAdvertencia('Verifica los nodos ingresados. Deben existir en la red.');
    return;
  }
  if (String(nodoOrig.id) === String(nodoDest.id)) {
    mostrarAdvertencia('El nodo inicial y el final no pueden coincidir.');
    return;
  }

  const peso = Number(pesoVal);
  if (pesoVal === '' || isNaN(peso) || !Number.isInteger(peso) || peso < 1 || peso > 9999) {
    mostrarAdvertencia('La capacidad debe ser un entero entre 1 y 9999.');
    return;
  }

  if (aristas.some(a => String(a.origen) === String(nodoOrig.id) && String(a.destino) === String(nodoDest.id))) {
    mostrarAdvertencia(`Ya existe una arista dirigida de ${nodoOrig.id} a ${nodoDest.id}.`);
    return;
  }

  aristas.push({ origen: nodoOrig.id, destino: nodoDest.id, peso, flujo: 0, isFictitious: false });
  if (seccionFord) seccionFord.style.display = 'none';
  if (grupoCorteMinimo) grupoCorteMinimo.innerHTML = '';

  dibujarRed();
  validarRed();
  mostrarToast(`Arista ${nodoOrig.id} → ${nodoDest.id} agregada`, 'exito');
}
```

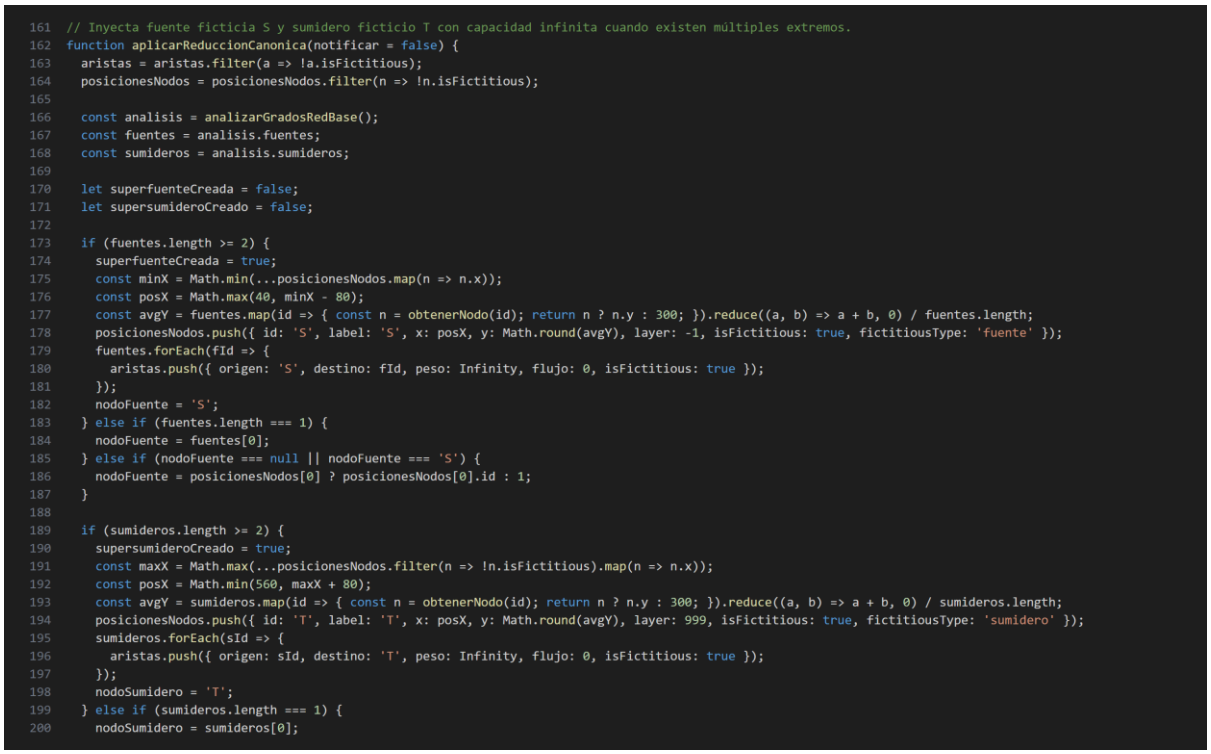
Nota. Función encargada de validar que los nodos existan, evitar lazos o aristas duplicadas, restringir la capacidad a enteros entre 1 y 9999 y registrar la arista dirigida con flujo inicial nulo. Elaboración propia.

Detección de Fuentes y Sumideros e Inyección de Nodos Ficticios

A partir de los grados de entrada y salida de cada vértice, el programa identifica las fuentes y sumideros reales de la red. Si existen dos o más de ellos, inyecta la fuente ficticia S o el sumidero ficticio T con aristas de capacidad infinita, que se representan con líneas discontinuas y la etiqueta flujo/∞.

Figura 3

Reducción de una red con múltiples fuentes y sumideros



```
// Inyecta fuente ficticia S y sumidero ficticio T con capacidad infinita cuando existen múltiples extremos.
function aplicarReduccionCanonica(notificar = false) {
  aristas = aristas.filter(a => !a.isFictitious);
  posicionesNodos = posicionesNodos.filter(n => !n.isFictitious);

  const analisis = analizarGradosRedBase();
  const fuentes = analisis.fuentes;
  const sumideros = analisis.sumideros;

  let superfuenteCreada = false;
  let supersumideroCreado = false;

  if (fuentes.length >= 2) {
    superfuenteCreada = true;
    const minX = Math.min(...posicionesNodos.map(n => n.x));
    const posX = Math.max(40, minX - 80);
    const avgY = fuentes.map(id => { const n = obtenerNodo(id); return n ? n.y : 300; }).reduce((a, b) => a + b, 0) / fuentes.length;
    posicionesNodos.push({ id: 'S', label: 'S', x: posX, y: Math.round(avgY), layer: -1, isFictitious: true, fictitiousType: 'fuente' });
    fuentes.forEach(fId => {
      aristas.push({ origen: 'S', destino: fId, peso: Infinity, flujo: 0, isFictitious: true });
    });
    nodoFuente = 'S';
  } else if (fuentes.length === 1) {
    nodoFuente = fuentes[0];
  } else if (nodoFuente === null || nodoFuente === 'S') {
    nodoFuente = posicionesNodos[0] ? posicionesNodos[0].id : 1;
  }

  if (sumideros.length >= 2) {
    supersumideroCreado = true;
    const maxX = Math.max(...posicionesNodos.filter(n => !n.isFictitious).map(n => n.x));
    const posX = Math.min(560, maxX + 80);
    const avgY = sumideros.map(id => { const n = obtenerNodo(id); return n ? n.y : 300; }).reduce((a, b) => a + b, 0) / sumideros.length;
    posicionesNodos.push({ id: 'T', label: 'T', x: posX, y: Math.round(avgY), layer: 999, isFictitious: true, fictitiousType: 'sumidero' });
    sumideros.forEach(sId => {
      aristas.push({ origen: sId, destino: 'T', peso: Infinity, flujo: 0, isFictitious: true });
    });
    nodoSumidero = 'T';
  } else if (sumideros.length === 1) {
    nodoSumidero = sumideros[0];
```

Nota. La función elimina nodos ficticios anteriores, analiza los grados de la red y agrega la fuente S y el sumidero T conectados con capacidad infinita (Infinity) cuando se detectan varios extremos. Elaboración propia.

Búsqueda del Camino Aumentante mediante BFS

El programa recorre el grafo mediante una búsqueda en anchura (BFS) desde el nodo fuente, avanzando únicamente por aristas con capacidad residual disponible. Al alcanzar el sumidero, reconstruye el camino encontrado a través del mapa de padres. Esta función se utiliza para validar la red y para detectar cuándo ya no existen más caminos de aumento.

Figura 4

Búsqueda de caminos con capacidad residual positiva

```
// Encuentra un camino aumentante residual entre la fuente y el sumidero usando BFS.
function buscarCaminoAumentanteBFS() {
  if (nodoFuente === null || nodoSumidero === null) return null;
  const parent = new Map();
  const queue = [nodoFuente];
  const visitado = new Set([String(nodoFuente)]);

  while (queue.length > 0) {
    const u = queue.shift();
    if (String(u) === String(nodoSumidero)) break;

    for (const a of aristas) {
      if (String(a.origen) === String(u)) {
        const destStr = String(a.destino);
        const residual = Number(a.peso) - Number(a.flujo);
        if (!visitado.has(destStr) && residual > 0) {
          visitado.add(destStr);
          parent.set(destStr, a.origen);
          queue.push(a.destino);
        }
      }
    }
  }

  if (!visitado.has(String(nodoSumidero))) return null;

  const camino = [];
  let curr = nodoSumidero;
  while (String(curr) !== String(nodoFuente)) {
    camino.unshift(curr);
    curr = parent.get(String(curr));
  }
  camino.unshift(nodoFuente);
  return camino;
}
```

Nota. La función identifica un camino desde la fuente hasta el sumidero con capacidad residual disponible y lo reconstruye; si devuelve null, el algoritmo ha finalizado. Elaboración propia.

Cálculo y Validación del Cuello de Botella

Una vez seleccionado y verificado el camino aumentante, el programa calcula la capacidad residual mínima entre las aristas que lo componen y solicita al usuario confirmarla. Si el valor ingresado coincide con el cuello de botella, se actualiza el flujo de cada arista y el flujo total de la red.

Figura 5

Validación del cuello de botella y actualización de flujos

```
// Confirma el cuello de botella, aumenta el flujo residual y comprueba la condición de corte mínimo.
function confirmarCapacidad() {
  const capIngresada = Number(inputCapacidadCamino.value.trim());

  let cuelloBotella = Infinity;
  for (let i = 0; i < caminoSeleccionado.length - 1; i++) {
    const u = caminoSeleccionado[i];
    const v = caminoSeleccionado[i + 1];
    const arista = aristas.find(a => String(a.origen) === String(u) && String(a.destino) === String(v));
    const residual = arista.peso - arista.flujo;
    if (residual < cuelloBotella) cuelloBotella = residual;
  }

  if (isNaN(capIngresada) || capIngresada !== cuelloBotella || capIngresada <= 0) {
    mostrarAdvertencia(`Capacidad incorrecta. Debe ser el cuello de botella (${cuelloBotella}) del camino.`);
    return;
  }

  for (let i = 0; i < caminoSeleccionado.length - 1; i++) {
    const u = caminoSeleccionado[i];
    const v = caminoSeleccionado[i + 1];
    const arista = aristas.find(a => String(a.origen) === String(u) && String(a.destino) === String(v));
    arista.flujo += cuelloBotella;
  }

  flujoTotal += cuelloBotella;
  historialRecorridos.push({ camino: caminoSeleccionado.join(' -> '), capacidad: cuelloBotella });
  valFlujoTotal.textContent = flujoTotal;

  caminoSeleccionado = [];
  textoCamino.textContent = 'Ninguno';
  btnVerificarCamino.disabled = true;
  divPreguntaCapacidad.style.display = 'none';

  dibujarRed();
  actualizarHistorialInteractivo();

```

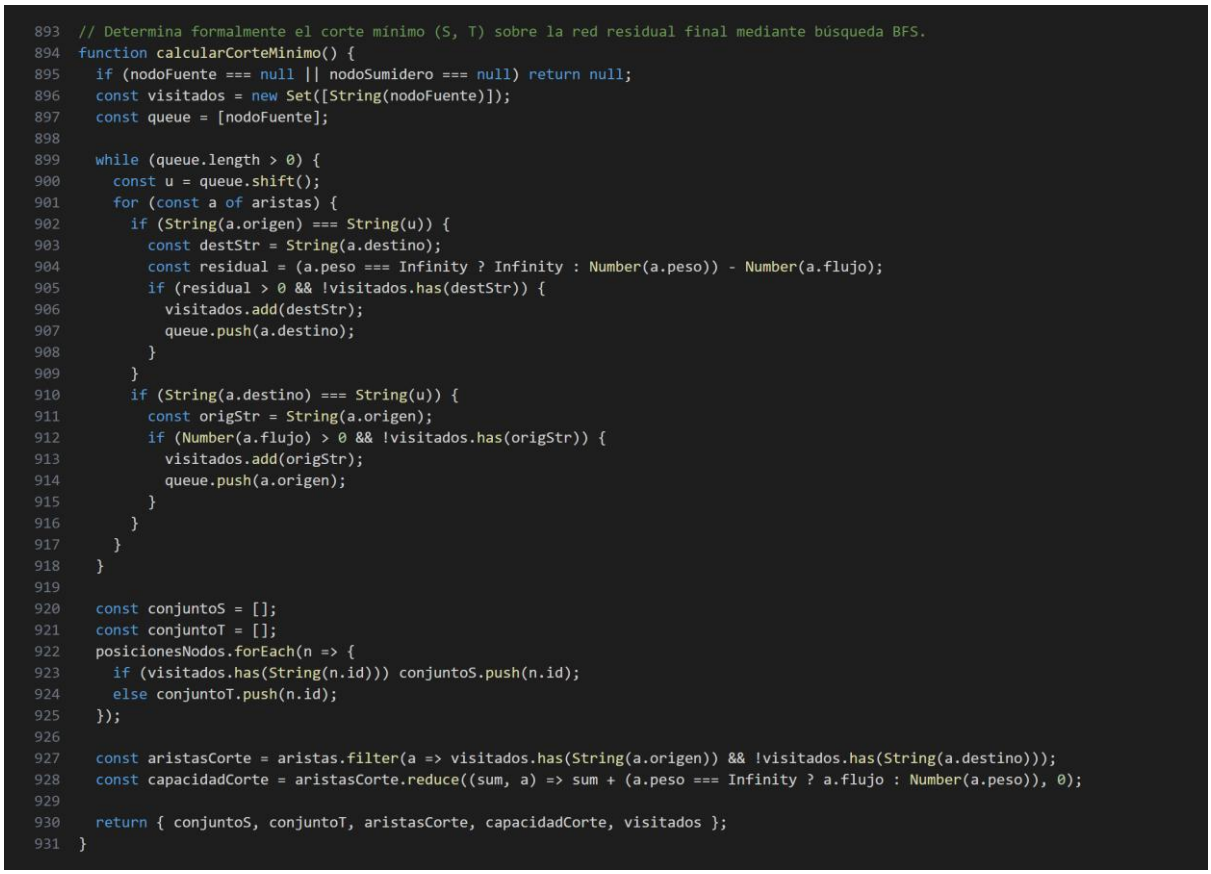
Nota. Calcula Δ como la capacidad residual mínima del camino, valida el valor ingresado por el usuario, actualiza el flujo de las aristas y registra el camino en el historial. Elaboración propia.

Cálculo del Corte Mínimo

Cuando no existen más caminos de aumento, el sistema realiza una búsqueda BFS sobre la red residual final, considerando tanto los arcos con capacidad libre como los arcos de retroceso con flujo positivo. Los vértices visitados conforman el conjunto S y los restantes el conjunto T; luego se suman las capacidades de las aristas que cruzan de S hacia T para comprobar que |f| = c(S, T).

Figura 6

Determinación del corte mínimo (S, T)



```
// Determina formalmente el corte mínimo (S, T) sobre la red residual final mediante búsqueda BFS.
function calcularCorteMinimo() {
  if (nodoFuente === null || nodoSumidero === null) return null;
  const visitados = new Set([String(nodoFuente)]);
  const queue = [nodoFuente];

  while (queue.length > 0) {
    const u = queue.shift();
    for (const a of aristas) {
      if (String(a.origen) === String(u)) {
        const destStr = String(a.destino);
        const residual = (a.peso === Infinity ? Infinity : Number(a.peso)) - Number(a.flujo);
        if (residual > 0 && !visitados.has(destStr)) {
          visitados.add(destStr);
          queue.push(a.destino);
        }
      }
      if (String(a.destino) === String(u)) {
        const origStr = String(a.origen);
        if (Number(a.flujo) > 0 && !visitados.has(origStr)) {
          visitados.add(origStr);
          queue.push(a.origen);
        }
      }
    }
  }

  const conjuntoS = [];
  const conjuntoT = [];
  posicionesNodos.forEach(n => {
    if (visitados.has(String(n.id))) conjuntoS.push(n.id);
    else conjuntoT.push(n.id);
  });

  const aristasCorte = aristas.filter(a => visitados.has(String(a.origen)) && !visitados.has(String(a.destino)));
  const capacidadCorte = aristasCorte.reduce((sum, a) => sum + (a.peso === Infinity ? a.flujo : Number(a.peso)), 0);

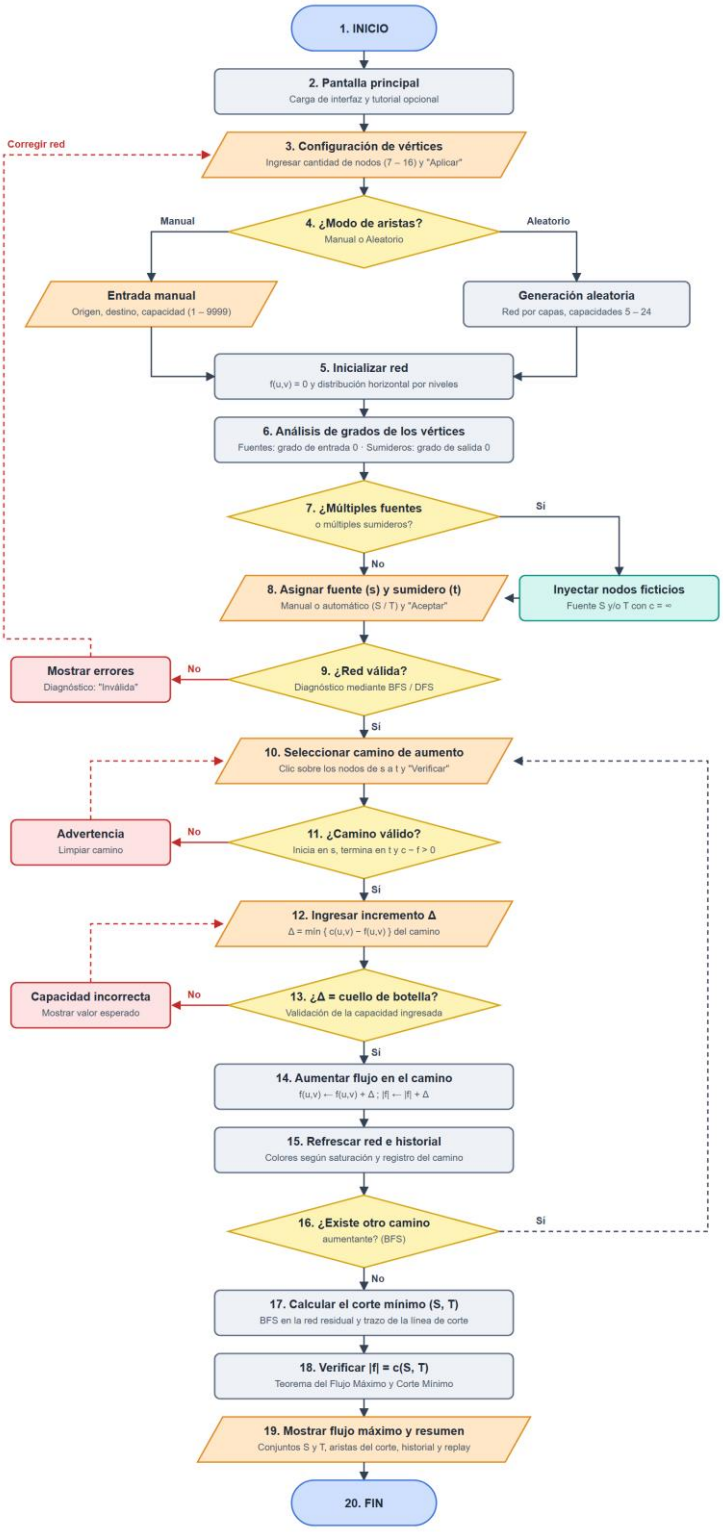
  return { conjuntoS, conjuntoT, aristasCorte, capacidadCorte, visitados };
}
```

Nota. La función devuelve los conjuntos S y T, las aristas del corte y su capacidad, que se muestran en el panel de resultados y se dibujan como una línea discontinua roja en el lienzo. Elaboración propia.

Diagrama de Flujo del Programa Ejecutable

Figura 7

Diagrama de flujo del sistema para la identificación del flujo máximo



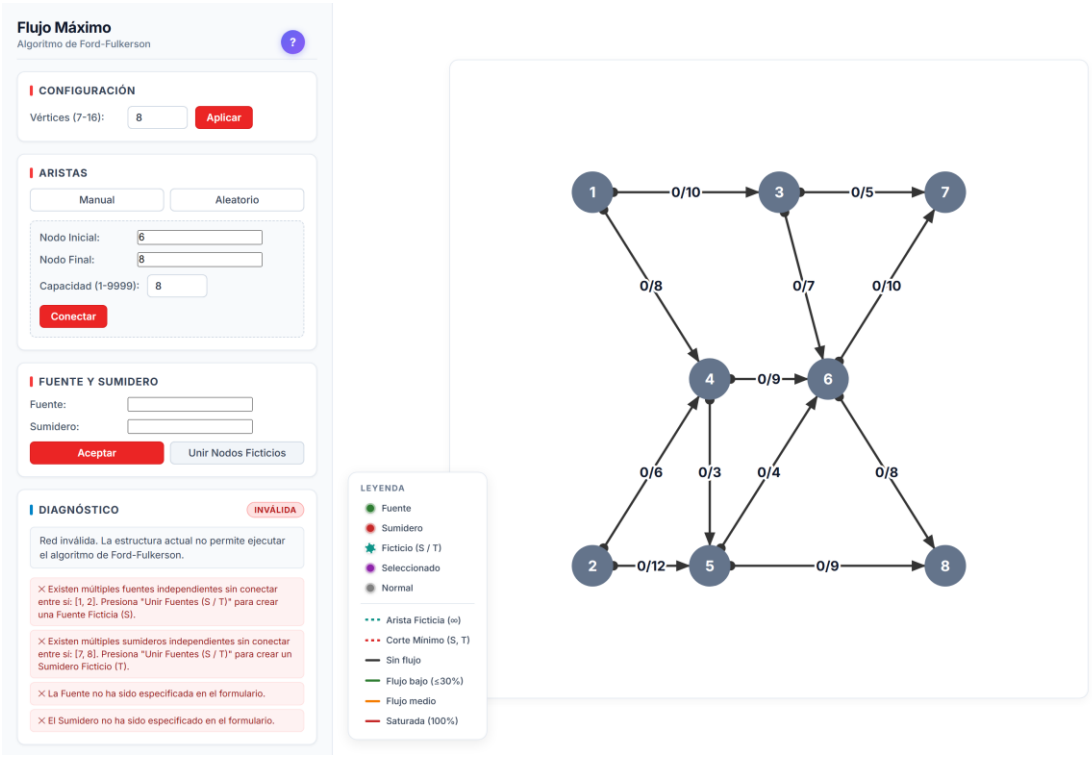
Nota. El diagrama muestra la secuencia de operaciones ejecutadas por la aplicación web, desde la configuración de vértices y aristas, la detección de múltiples fuentes y sumideros con la inyección de nodos ficticios, la validación interactiva de caminos de aumento y del cuello de botella, hasta el cálculo del corte mínimo y la verificación $|f| = c(S, T)$. Elaboración propia.

Visualización de Resultados

Los resultados se muestran al usuario de forma gráfica y textual. El lienzo SVG representa cada arista con la etiqueta flujo/capacidad y un código de colores según su saturación: negro cuando no tiene flujo, verde cuando la saturación es menor o igual al 30 %, naranja para una saturación media y rojo cuando la arista está saturada. La fuente se muestra en verde, el sumidero en rojo, los nodos ficticios con borde discontinuo y los nodos del camino seleccionado en morado. Asimismo, el panel de Diagnóstico informa si la red es válida o lista sus errores, como nodos aislados, callejones sin salida o múltiples fuentes sin unificar.

Figura 8

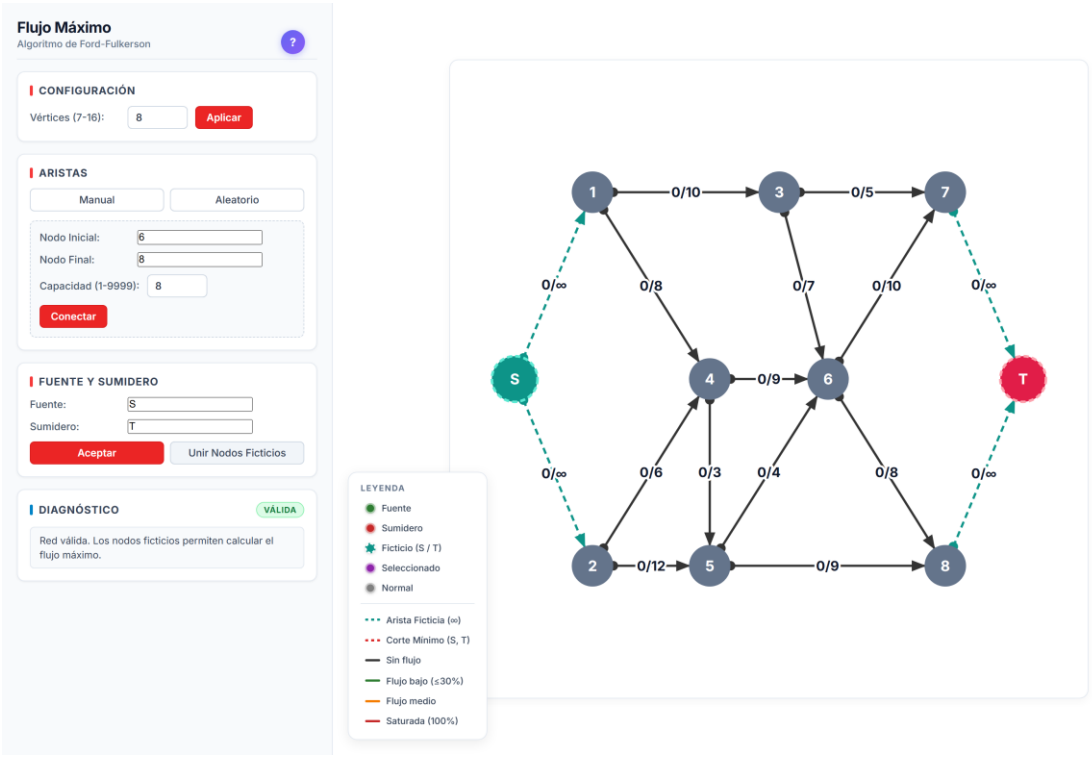
Diagnóstico de una red con múltiples fuentes y sumideros



Nota. La red ingresada tiene dos fuentes (1 y 2) y dos sumideros (7 y 8); el panel de Diagnóstico la marca como inválida y sugiere crear la fuente y el sumidero ficticios. Elaboración propia.

Figura 9

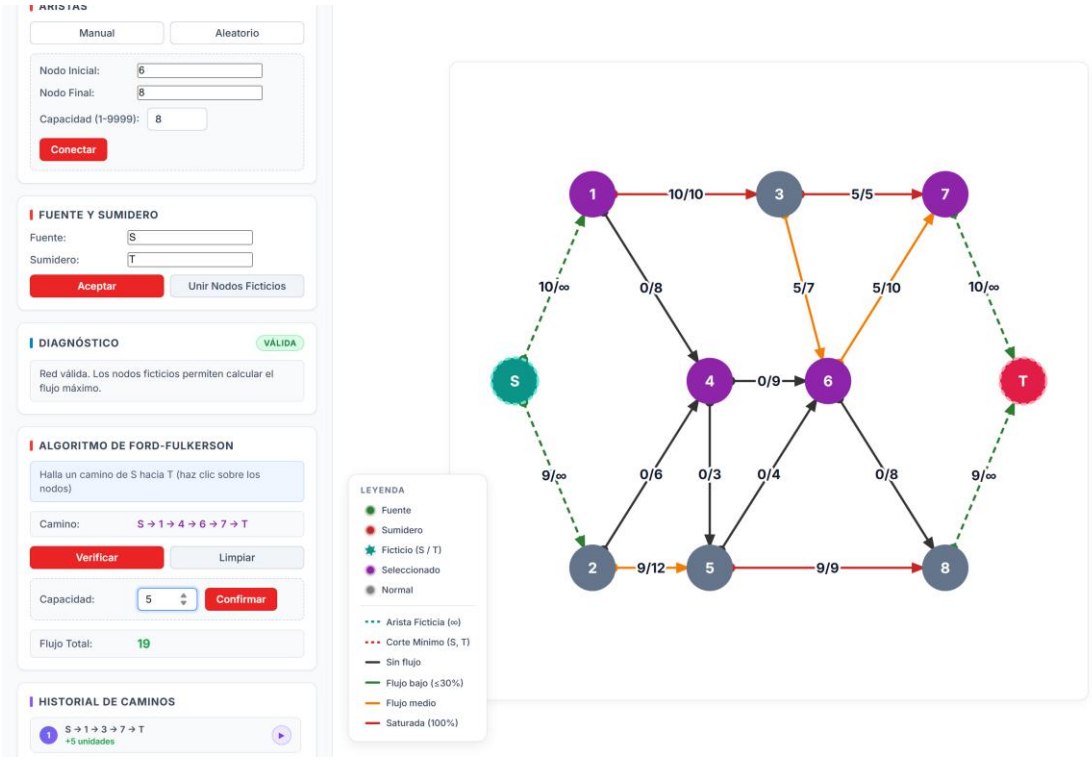
Red transformada con fuente ficticia S y sumidero ficticio T



Nota. Tras presionar "Unir Nodos Ficticios", el sistema agrega S y T con aristas de capacidad infinita (líneas discontinuas) y la red pasa a ser válida. Elaboración propia.

Figura 10

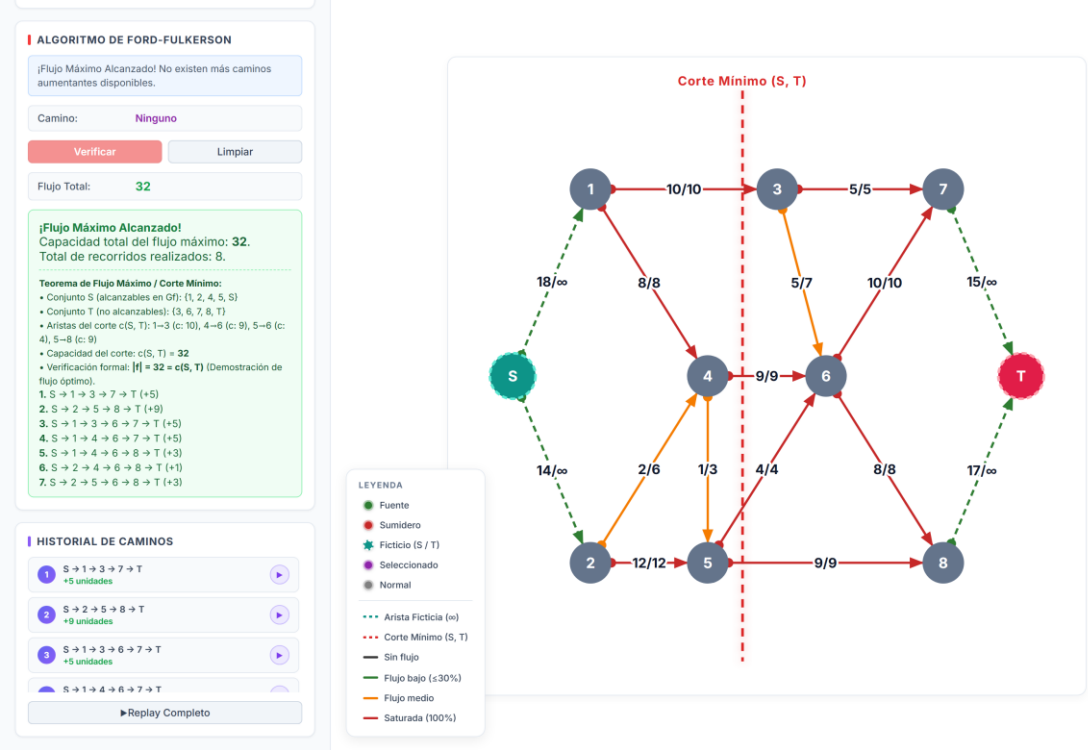
Selección interactiva de un camino de aumento



Nota. El usuario selecciona el camino S → 1 → 4 → 6 → 7 → T (nodos en morado) e ingresa el cuello de botella Δ = 5 antes de confirmar; el flujo acumulado hasta ese momento es 19. Elaboración propia.

Figura 11

Resultado final con el flujo máximo y el corte mínimo



Nota. El programa informa el flujo máximo |f| = 32, los conjuntos S y T, las aristas del corte, la verificación |f| = c(S, T) y el historial de caminos con opción de reproducción. Elaboración propia.

Programa

Se presenta la versión final del simulador interactivo web (HTML5, CSS y JavaScript) para el algoritmo de Ford-Fulkerson. El sistema permite construir redes de 7 a 16 vértices de forma manual o aleatoria, valida la estructura mediante un panel de diagnóstico, integra automáticamente orígenes y destinos ficticios cuando existen múltiples fuentes o sumideros, permite seleccionar caminos de aumento paso a paso y, al finalizar, calcula y dibuja el corte mínimo verificando el Teorema del Flujo Máximo y Corte Mínimo. Además, incorpora un tutorial guiado, notificaciones, una leyenda interactiva, información de cada nodo al pasar el cursor y un historial de caminos con reproducción animada. La plataforma desarrollada y su código fuente están disponibles en los siguientes enlaces:

- Despliegue funcional: https://flujo-maximo-sem7-grupo-9.netlify.app/
- Repositorio y control de versiones: https://github.com/lincolnangae/FlujoMaximo-Ford-F-Grupo-9

Resultados

Para verificar el correcto funcionamiento del programa se resolvió un caso de prueba con 8 vértices, dos fuentes (vértices 1 y 2) y dos sumideros (vértices 7 y 8). Las capacidades de cada arista, expresadas en unidades de flujo, se muestran en la Tabla 1.

Tabla 1

Capacidades de las aristas de la red de prueba

Arista	Capacidad	Arista	Capacidad
1 → 3	10	3 → 7	5
1 → 4	8	4 → 5	3
2 → 4	6	4 → 6	9
2 → 5	12	5 → 6	4
3 → 6	7	5 → 8	9
6 → 7	10	6 → 8	8

Interpretación y Representación

Como la red posee múltiples fuentes y sumideros, el programa agrega la fuente ficticia S, con aristas S → 1 y S → 2 de capacidad infinita, y el sumidero ficticio T, con aristas 7 → T y 8 → T de capacidad infinita. Se definen las variables c(u, v) como la capacidad de cada arco y f(u, v) como el flujo que circula por él, con $0 \leq f(u, v) \leq c(u, v)$, partiendo de f(u, v) = 0 en todos los arcos.

Cálculo

Se aplicó el algoritmo de Ford-Fulkerson seleccionando en el programa los caminos de aumento que se resumen en la Tabla 2. En cada iteración se calculó el cuello de botella como la menor capacidad residual del camino.

Tabla 2

Iteraciones del algoritmo de Ford-Fulkerson en la red de prueba

Iteración	Camino de aumento	Capacidades residuales	Δ	|f|
1	S → 1 → 3 → 7 → T	∞, 10, 5, ∞	5	5
2	S → 2 → 5 → 8 → T	∞, 12, 9, ∞	9	14
3	S → 1 → 3 → 6 → 7 → T	∞, 5, 7, 10, ∞	5	19
4	S → 1 → 4 → 6 → 7 → T	∞, 8, 9, 5, ∞	5	24
5	S → 1 → 4 → 6 → 8 → T	∞, 3, 4, 8, ∞	3	27
6	S → 2 → 4 → 6 → 8 → T	∞, 6, 1, 5, ∞	1	28
7	S → 2 → 5 → 6 → 8 → T	∞, 3, 4, 4, ∞	3	31
8	S → 2 → 4 → 5 → 6 → 8 → T	∞, 5, 3, 1, 1, ∞	1	32

Nota. Las capacidades residuales c(u, v) − f(u, v) se listan en el orden de las aristas del camino. Elaboración propia.

Por ejemplo, en la cuarta iteración el camino p = S → 1 → 4 → 6 → 7 → T tiene como capacidades residuales r(1, 4) = 8 − 0 = 8, r(4, 6) = 9 − 0 = 9 y r(6, 7) = 10 − 5 = 5, por lo que:

$$\Delta = \text{mín}\{\infty, 8, 9, 5, \infty\} = 5$$

Luego de la octava iteración, la búsqueda BFS ya no encuentra ningún camino de S a T con capacidad residual positiva, por lo que el algoritmo finaliza con un flujo total |f| = 32. La red de flujos final cumple la restricción de capacidad en todas las aristas y la conservación del flujo en cada vértice intermedio; por ejemplo, en el vértice 6 ingresan 5 + 9 + 4 = 18 unidades y salen 10 + 8 = 18 unidades.

Análisis y Argumentación

Al finalizar el algoritmo, el programa identifica el corte S = {S, 1, 2, 4, 5} y T = {3, 6, 7, 8, T}, cuyas aristas de S hacia T son 1 → 3, 4 → 6, 5 → 6 y 5 → 8. Su capacidad es:

$$c(S, T) = c(1, 3) + c(4, 6) + c(5, 6) + c(5, 8) = 10 + 9 + 4 + 9 = 32$$

Como el valor del flujo obtenido es |f| = 32 y se cumple que |f| = c(S, T), se concluye, de acuerdo con el Teorema del Flujo Máximo y Corte Mínimo, que el flujo máximo de la red

es 32 unidades. Este resultado coincide con el reportado por el programa en la Figura 11 y con el cálculo realizado manualmente, lo que valida la implementación. Además, las aristas del corte, todas saturadas, representan los cuellos de botella de la red: para aumentar el flujo máximo sería necesario ampliar la capacidad de alguna de ellas.

Durante las pruebas se observó que el programa facilita significativamente el estudio de las redes de flujo, ya que automatiza un proceso que resulta laborioso cuando se realiza manualmente. La validación del cuello de botella, el código de colores y el historial de caminos permiten detectar errores de cálculo de forma inmediata y analizar distintas configuraciones de red sin repetir operaciones. Asimismo, la transformación automática con orígenes y destinos ficticios permitió resolver redes con varios extremos de la misma forma que una red con una sola fuente y un solo sumidero.

Conclusiones

El desarrollo del proyecto permitió comprender los fundamentos de las redes de flujo y del algoritmo de Ford-Fulkerson, así como su aplicación en problemas de transporte y distribución de recursos. A través del estudio de las capacidades residuales y de los caminos de aumento, fue posible analizar cómo el flujo de una red se incrementa progresivamente hasta alcanzar su valor máximo.

La implementación computacional facilitó la verificación del resultado mediante el Teorema del Flujo Máximo y Corte Mínimo. En el caso de prueba, el flujo máximo obtenido (32 unidades) coincidió con la capacidad del corte mínimo identificado por el programa, lo que demuestra que la solución es óptima y que el corte permite reconocer las aristas que limitan la capacidad de la red.

La incorporación de orígenes y destinos ficticios con capacidad infinita permitió extender el programa a redes con múltiples fuentes y sumideros sin alterar su flujo máximo, superando la limitación de la versión preliminar, que solo admitía una fuente y un sumidero.

Finalmente, el proyecto evidenció la importancia de las herramientas computacionales como apoyo para el análisis de problemas matemáticos. El uso de JavaScript, HTML5 y SVG permitió desarrollar una herramienta didáctica capaz de representar, validar y visualizar el proceso de optimización de manera interactiva, contribuyendo al aprendizaje de los conceptos propios de la Matemática Computacional.

Referencias Bibliográficas

Ahuja, R. K., Magnanti, T. L. & Orlin, J. B. (1993). *Network flows: Theory, algorithms, and applications*. Prentice Hall.

Cormen, T. H., Leiserson, C. E., Rivest, R. L. & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). The MIT Press.

Departamento de Ciencias. (2026). *Lectura 6.1: Redes de flujo* [Material de clase]. Universidad Peruana de Ciencias Aplicadas.

Departamento de Ciencias. (2026). *Lectura 6.2: Algoritmo de Ford Fulkerson* [Material de clase]. Universidad Peruana de Ciencias Aplicadas.

Edmonds, J. & Karp, R. M. (1972). Theoretical improvements in algorithmic efficiency for network flow problems. *Journal of the ACM, 19*(2), 248-264.

Ford, L. R. & Fulkerson, D. R. (1956). Maximal flow through a network. *Canadian Journal of Mathematics, 8*(3), 399-404.

Kleinberg, J. & Tardos, É. (2006). *Algorithm design*. Pearson Addison-Wesley.

Rosen, K. H. (2019). *Discrete mathematics and its applications* (8.ª ed.). McGraw-Hill Education.